

AI Business Automation Platform — Project Documentation

Autor: bryan | Data: 01/10/2026

AI Business Automation Platform

Project Documentation · Generated 2026-10-01 · Version 1.0

1. Overview

A full-stack support-desk automation platform: customers submit tickets, an AI pipeline classifies and drafts a reply, and an administrator confirms it before anything is sent.

The product's positioning claim is deliberately narrow and defensible: ****AI output is never allowed to become a human answer on its own.**** Every analysis carries an explicit provenance state, and the interface makes that state impossible to mistake. Combined with a capped, savepoint-isolated workflow engine that records exactly one honest run row, the claim is ***automation you can audit***, not automation you must trust.

Users

Administrator — the person running the business's support operation. Sits in the dashboard repeatedly through the day: triaging new tickets, reviewing what the AI concluded, confirming or overriding it, replying to customers, and adjusting automation rules. Success is a ticket correctly routed, correctly answered, with a human having actually looked at the AI's work.

Customer — a person with a problem, contacting support. Usually one visit, often mildly stressed. Success is submitting once and watching it progress without re-explaining themselves.

Technology Stack

| Layer | Choice |

| --- | --- |

Frontend	Vite + React 19 + TypeScript + Tailwind CSS 4 + React Router 7
Icons / type	lucide-react (1.75 stroke) - Archivo Variable - Courier Prime
Backend	FastAPI + Pydantic settings + SQLAlchemy 2 + Alembic
Database	PostgreSQL (Supabase), session pooler on port 5432
Auth	JWT (Bearer), bcrypt via passlib, admin role gated server-side
AI	Provider interface: mock (default) - ollama (cloud or local)
Tests	pytest (78 passed + 1 skipped) - ESLint + tsc build gate

2. Architecture

Ticket creation returns immediately; AI analysis and workflow evaluation run as background steps, each opening its own database session.



Layer responsibilities

The backend is authoritative for all validation and authorization. Two roles exist only: customer and admin, both enforced server-side. The frontend never calls the AI provider directly — AI credentials never reach the browser, and the AI has no HTTP surface at all.

3. Feature Set

3.1 Customer portal (/portal)

- Signup and login (JWT in localStorage, sent as Authorization: Bearer header)
- Role-aware redirects after authentication
- Submit tickets and track only their own tickets

- Read status and priority; reply to the support thread
- AI analysis runs automatically after submission — the customer sees a suggested reply drafted for a human to confirm

3.2 AI analysis pipeline

- One contract, swappable engines:
- **mock** — deterministic keyword rules, offline, no dependencies. This is the default used by all tests and seeded demo data.
- **ollama** — real model via the Ollama API, cloud or local install
- Classifies category, priority and sentiment
- Drafts a summary and a suggested response
- Writes category and priority back to the ticket
- **Failures can never break ticket creation.** Any provider or validation error is persisted as a failed analysis with a sanitized message; the ticket is unaffected.

3.3 Admin console (/admin, admin role required)

- **Ticket line** — a five-station rail reading NEW, ANALYZE, REVIEW, REPLY, CLOSED, with recent carriers and one primary action
- Cross-customer ticket management with status and priority filters
- Triage queue filter separating tickets that await human confirmation
- Status changes, tags, and replies on both sides
- AI reply review with human confirmation (is_human_confirmed), sealed with the reviewer's initials and timestamp
- Customers list (read-only)
- Workflows CRUD with run history
- Analytics over 7/30/90-day windows, UTC-day buckets, zero-filled

3.4 Workflow engine (capped by design)

- Single trigger: ticket.created
 - AND-only conditions, maximum 5: category is, priority is, sentiment is, status is
 - Up to 5 actions from 4 types: set_priority, add_tag, generate_suggested_response, record_notification
 - UI is configuration forms, not a visual builder
 - **Guarantee:** exactly one workflow_runs row per evaluation, recorded as success, skipped, or failed. Failing actions roll back via savepoint, so a half-applied automation leaves no trace beyond its failed run.
 - The engine never raises into a request.
-

4. Design System

The visual language is a **pneumatic-tube dispatch desk** — a mid-century dispatch office with an aluminium surface, stations engraved into it, and carriers running a single continuous line.

4.1 The palette law

Six hues, one meaning each. A hue is never borrowed for emphasis, decoration, or variety.

Hue	Meaning
---	---
route (blue)	needs a human
ink	a human marked it
signal (amber)	a machine has it
fault (red)	failed or adverse
hazard (yellow)	blocked or irreversible
panel (neutral)	idle or none

There is **no green** in the system. Success is not a tint — a completed action is *settled*, so it renders as an ink-outlined plate.

4.2 Priority is a count, never a hue

Priority is expressed as a count of signal bands out of four, with unfilled slots drawn as ghosts. Urgency can therefore never be confused with a status colour.

4.3 Type carries authorship

If the machine printed it, it is set in Courier Prime. If a person wrote it, it is set in Archivo. Authorship and typeface must never disagree: customer messages, operator replies and notes stay Archivo; the AI's verdict and suggested response stay Courier.

4.4 Provenance is triple-encoded

- Dashed border + grey Machine head = unsealed machine output
- Solid border + blue Human strip = confirmed
- The ink band striking across the record (380ms) is the only authored motion in the entire system

4.5 Layout and depth

Single-column working surfaces on a max-w-5xl container (admin) or max-w-3xl (portal). Depth is carried mostly by *engraving* rather than lift: grooves (a 1px dark lip plus a 1px white highlight beneath) are the default separator. Shadows are ink-tinted, always offset, never a zero-offset halo, and appear only to seat a plate or raise the head row of a list.

5. Getting Started

5.1 Prerequisites

- Python 3.12+
- Node 20+ (tested on 24)
- A Supabase project (any PostgreSQL connection works)
- Optional for real AI analysis: an Ollama API key or a local install with a pulled model

5.2 Backend

```
%❏ POWERSHELL
cd backend
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
copy .env.example .env          # fill in DATABASE_URL + JWT_SECRET_KEY
.venv\Scripts\python -m alembic upgrade head
.venv\Scripts\uvicorn app.main:app --reload
```

Interactive API docs: <http://localhost:8000/docs>

Supabase note: use the direct or session connection on port 5432. The transaction pooler on 6543 rejects DDL and breaks Alembic migrations. The session pooler identifies the project via the username, which must be postgres.PROJECT-REF rather than bare postgres.

5.3 Frontend

```
%❏ POWERSHELL
cd frontend
npm install
npm run dev
```

App: <http://localhost:5173> — the dev server proxies /api to the backend on port 8000.

5.4 Demo accounts

```
| Account | Email | Password |  
| --- | --- | --- |  
| Administrator | admin@demo.example.com | Demo!Pass123 |  
| Customer | ava.reyes@demo.example.com | Demo!Pass123 |
```

Everything the seeder creates is tagged `is_demo` and is removed with `python -m app.cli.seed_demo --remove`. Real data is never touched.

Public registration never grants admin. To create a real administrator:

```
%❏ POWERSHELL  
.venv\Scripts\python -m app.cli.create_admin --email you@example.com --password strong-  
password
```

5.5 Real AI analysis (optional)

```
%❏ INI  
# backend/.env  
AI_PROVIDER=ollama  
OLLAMA_MODEL=gpt-oss:20b  
OLLAMA_API_KEY=your-key-here
```

For a fully local, unlimited alternative, install Ollama, run `ollama pull llama3.2`, then set `OLLAMA_BASE_URL` to `http://localhost:11434` and leave `OLLAMA_API_KEY` empty.

6. Verification

```
%❏ POWERSHELL  
# Backend - 78 passed, 1 skipped (Ollama live test skips when unconfigured)  
cd backend  
.venv\Scripts\python -m pytest -q  
  
# Frontend  
cd frontend  
npx tsc --noEmit  
npm run lint  
npm run build
```

Verification is lint, typecheck and manual or browser walkthrough — there is no frontend test framework. Authorization tests are mandatory for every protected route.

7. Project Structure

```
%T CODE
backend/
  app/
    api/          # routers: auth, tickets, admin, workflows
    ai/           # provider abstraction: mock.py, ollama.py, schemas.py
    workflows/    # capped automation engine
    services/     # analysis, analytics, workflow services
    schemas/      # Pydantic request/response contracts
    models/       # SQLAlchemy models + enums
    cli/          # create_admin, seed_demo
    core/         # config, security
  alembic/        # migrations
  tests/         # 78 tests
frontend/
  src/
    pages/        # home, auth, portal/, admin/
    features/     # auth context + route guards
    services/     # typed API client
    components/   # shared UI primitives
    index.css     # @theme tokens + component layer
```

8. Key Decisions

| # | Decision | Rationale |

| --- | --- | --- |

| D1 | Customer accounts with login | Demonstrates real role-based access control; avoids an email-provider dependency |

| D2 | Mock-first AI provider abstraction | Keeps every milestone runnable offline and free; the real provider slots in behind the same interface |

| D4 | JWT in localStorage + Bearer header | Consistent with the v1 API; an HTTP-only cookie move was deferred to a hardening pass |

| D6 | Supabase-hosted PostgreSQL | Managed, provisioned, and reachable from the dev network |

| - | Admin reply requires its own endpoint | The customer message route is ownership-checked and hardcodes sender=CUSTOMER, so widening it would have mislabelled admin replies |

| - | Seal records who signed it | Stamping initials requires persisted provenance; guessing from the logged-in admin is wrong whenever a second admin views the record |

9. Product Principles

1. **Provenance over polish** — an AI result must always be visually distinguishable from a human-confirmed one. If the two look alike, the design failed regardless of how it looks.

2. **Honest state** — failed and skipped things are shown as failed and skipped. Nothing renders as success it did not earn.

3. **Automation stays capped and readable** — the workflow engine is config, not

a programming language, and the UI never implies more power than exists.

4. **The operator's time is the resource** — scanability and in-place recovery beat novelty.

5. **Demo-able without a real business** — every screen looks complete with seeded data and does not depend on real customer volume.

10. Scope Boundaries

Deliberately excluded from v1: real email or push notifications (notifications are recorded events only), background job infrastructure such as Redis or Celery, a visual workflow builder, file uploads, and production deployment. The project is a portfolio demonstration; the verification rituals are pytest, ESLint, tsc, and live walkthroughs.